

Damn Vulnerable ICS Firmware

ICS / OT Firmware Reverse-Engineering Lab

What this is

Three deliberately vulnerable firmware images modelled on common ICS/OT devices: a protocol gateway, an embedded Linux PLC, and a protection relay / IED. Each is a real extractable image with planted findings. Everything you need is inside the file: no device, no network, no runtime. This is pure static firmware triage, the same first pass you'd run on any embedded target.

You'll get three files: **image1.bin**, **image2.bin**, **image3.bin**, numbered easiest to hardest. Working out what each one actually is (step 1 below) is the first task.

Findings come in a few shapes: a distinctive {token} that names what it is, a **credential or key** (password, hash, private key, community string), or a **software version** you look up for known CVEs. Each image has 8 flags, consisting of 6 that are unique to that image plus 2 which are present in all three images (find once and it counts everywhere). Collect as many as you can.

Ground rules

- Static only: work entirely from the image. Nothing here needs to be run.
- Difficulty ramps up with each image (image1 easiest, image3 hardest).
- Stuck? The hint ladder at the back gives graded nudges.
- Note *where* you found each thing and *how*. The method matters more than the flag.

Your toolkit

You will need a Linux machine (VM or WSL) with these installed, all free/open-source. See the **Setup** section below for explicit Ubuntu install commands.

Extraction	binwalk, unblob, sasquatch (squashfs), jefferson (jffs2), cpio
Triage	file, strings, grep/ripgrep, xxd / hexdump
Decode / crypto	CyberChef (browser), base64, john & hashcat + rockyou.txt
Binaries	radare2 or Ghidra, checksec, readelf, nm
Data	sqlite3, a hex editor (ImHex / GHex)

Setup (Ubuntu)

A clean Ubuntu 22.04 / 24.04 machine. Copy-paste to get a working kit; everything here is free.

1 • Extraction, triage & RE (apt)

```
sudo apt update
sudo apt install -y binwalk squashfs-tools cpio sqlite3 xxd binutils \
    git python3-pip pipx radare2 checksec john hashcat default-jdk
```

2 • Filesystem extractors that aren't in apt (pipx, isolated to avoid dependency clashes)

```
pipx install jefferson      # JFFS2 extractor
pipx install unblob        # optional all-in-one; auto-detects any filesystem
```

If jefferson errors with `Invalid attribute: hdr_crc_match`, its dependencies have drifted (common on rolling distros); the isolated pipx install above avoids that. If the command isn't found after install, run `pipx ensurepath` and reopen the shell, or call it by full path: `~/.local/share/pipx/venvs/jefferson/bin/jefferson`.

3 • Ghidra, hex editor, decoder

```
# Ghidra:      download + unzip from https://ghidra-sre.org (uses the JDK above)
sudo apt install -y ghex                                # GUI hex editor
# ImHex:       https://imhex.werwolv.net                 (standalone, nicer)
# CyberChef:   https://gchq.github.io/CyberChef           (base64/XOR decoding in a browser)
```

The 5-step triage workflow

Try these in order on every image. Most flags fall out of steps 3–5.

1 • Identify & extract

`file imageX.bin` tells you the filesystem. You don't need to know the matching unpacker off by heart: `binwalk -e imageX.bin` (or `unblob imageX.bin`) auto-detects it, calls the right extractor and handles nested layers. The table below is the filesystem→tool map if you'd rather do it by hand. Note the extracted root, which is your working directory from here.

2 • Map the filesystem

`find . -type f`. Get oriented: `/etc` (configs), `/usr/sbin` & `/opt` (binaries and apps), `/var` (data, databases). Skim the layout before diving in.

3 • Low-hanging fruit

`grep` configs for `pass` / `secret` / `key` / `community`; read every file in `/etc`; `strings` the text files. Then look again at the files that *aren't* obviously config: backups, dot-files, odd extensions.

4 • Binaries

Find versions (banners, `*.version` files, `--version` strings) → look up CVEs. Run `checksec` on network-facing binaries. Then `strings`, and when that runs out, open it in Ghidra/radare2. On a static binary `strings` also dumps all of `glibc`, so `readelf -p .rodata` narrows it to the binary's own constant strings, where hardcoded endpoints and keys live.

5 • The leftovers

Data files (.dat), databases (sqlite3 file.db '.dump'), ICS logic (.st). And the one people miss: compare the **image size** to the extracted size. If the image is much bigger, something lives outside the filesystem. dd it out and look.

Extractors by filesystem

binwalk -e and unblob pick the right one for you; this table is so you understand what they're doing, and what to reach for by hand if a front-end struggles.

file reports...	Unpack it by hand with...
Squashfs	unsquashfs imageX.bin (sasquatch for odd vendor variants)
JFFS2	jefferson imageX.bin -d out/
ASCII cpio (newc)	cpio -idv < imageX.bin (run inside an empty directory)
ext2 / ext3 / ext4	sudo mount -o loop imageX.bin /mnt (or debugfs)
not sure / mixed	unblob imageX.bin auto-detects & unpacks everything, nested too

A worked micro-example

Not one of the lab flags, just the rhythm of steps 1–3 so you know what 'good' looks like:

```
$ file router.bin
router.bin: Squashfs filesystem, little endian, version 4.0
$ unblob router.bin
$ find router.bin_extract -type d -name etc # the real rootfs is buried under an offset-named
dir
router.bin_extract/0-2555904.squashfs_extract/etc
$ cd router.bin_extract/0-2555904.squashfs_extract
$ grep -rniE 'pass|secret|community' etc/
etc/snmp/snmpd.conf:12: rocommunity public
$ ls -la etc/*.bak etc/*.old 2>/dev/null
etc/config.bak ← read this; scrubbed live configs often survive here
```

What to expect

The flags span a wide range. The easiest are a single grep or strings away; the hardest need disassembly (Ghidra/radare2), correlating content across files, or spotting data hidden outside the filesystem. Clear the quick wins across all three images first, then push into the harder ones; nobody's expected to get everything in a morning.

Hint ladder

Escalating nudges: read only as far down as you need. They point at the *method*, not the answer.

image1.bin: Protocol / Serial Gateway

- 1 This one is config-heavy; most of it lives in /etc.
- 2 grep recursively for credential-ish keywords, and don't stop at .conf files.
- 3 One secret hides in a backup copy; one 'flag' is a hash you must crack; and two of the network services are outdated, so fingerprint their versions from the service binaries themselves and look each up.

image2.bin: Embedded Linux PLC

- 1 Run `readelf -p` on the executables you extracted. The PLC's control runtime (its name gives it away) has something in its `.rodata` about how it phones home.
- 2 Not everything is plaintext: one value is assembled at runtime (reach for a disassembler), another is base64 over something.
- 3 ICS logic files (`.st`) are plain readable text, so open them.

image3.bin: Protection Relay / IED

- 1 Enumerate the binaries and the data files first.
- 2 Two files might only mean something combined; a database hides an account.
- 3 The image is bigger than the filesystem inside it, so find where the filesystem ends, and look at what sits after it (don't just du the extraction folder; carvers leave a whole-image copy that inflates it).

Where to learn more

- **OWASP Firmware Security Testing Methodology (FSTM)**: the 9-stage methodology this lab is a hands-on slice of.
- **OWASP IoT Top 10**: the categories each flag maps to (I1 weak/default creds, I2 insecure network services, I5 outdated components, I7 insecure data storage, I9 insecure defaults, I10 lack of hardening).
- **Repository & updates**: github.com/TartarusLabs/damn-vulnerable-ics-firmware